Verify, Don't Decode: A Trusted Monitor for Steganographic Collusion in Multi-Agent Software Teams and Markets

Muni*

October 2026 — sprint report, v0.1

Abstract

Colluding LLM agents can hide their coordination from an overseer using steganography: codebook phrases, signals in code artifacts, tool-mediated side channels, or the trailing digits of prices. Monitors that read transcripts are therefore fragile. We build a *trusted monitor* that does not try to decode the hidden channel. Instead, it independently verifies the outcome the channel is meant to bring about, and it tracks information-flow provenance to attribute coordination to specific agents. We evaluate it in two real-world settings behind one interface. (B) is a five-role agent team implementing real FastAPI features from five upstream commits merged on 21 Oct 2026, with hidden SWE-bench-style acceptance tests. (A) is a continuous double auction framed as a wholesale power exchange, with harm measured against the competitive equilibrium. Across ten software and seven market conditions, with hard-negative controls for legitimate teamwork and benign price convergence, a model-free verification-and-provenance monitor detects harmful coordination for every channel we tested. Its detection does not change when chat transcripts are removed. A rules-only transcript baseline, by contrast, sees only overt collusion. Detection is channel-invariant; attribution is not. A trusted competitive entrant restores consumer surplus without harming benign markets, whereas quarantining suspected cartel members backfires. The v0.1 numbers come from seeded scripted agents with exact ground truth. The LLM-agent pipeline is implemented and awaits API runs. We report the gaps this leaves.

1 Introduction

Multi-agent LLM deployments rely on *role separation*: a tester checks the implementer, a reviewer checks both, and sellers compete. Collusion breaks that separation. Prior work shows that LLM agents can hide coordination steganographically [14], that such schemes can emerge under misspecified rewards and survive paraphrasing [13], and that tool use makes them easier to build [15]. In markets, supra-competitive prices arise even without communication [1, 5, 8]. Together these results undermine the default oversight design: another LLM reads the agents' messages and asks whether they look suspicious.

We take the opposite stance. The monitor should not have to prove *how* agents coordinated before it blocks a demonstrably unsafe release or a market outcome. Our trusted monitor (i) observes an authoritative, hash-chained environment log that agents cannot write to; (ii) verifies outcomes with evidence the agents do not control (hidden tests on a pristine checkout, mutation testing of the team's own tests, competitive-equilibrium benchmarks); (iii) uses coordination evidence only for investigation and attribution; and (iv) intervenes in graduated steps (ALLOW/HOLD/CONSTRAIN). Our contributions:

- A two-domain harness on *real* artifacts. The software scenario uses five FastAPI changes merged after current models' training cut-off. The market scenario is a CDA following Agrawal et al. [3]. Both scenarios include steganographic, leakage, tacit and hard-negative conditions with separate mechanism and harm labels.
- A domain-agnostic monitor interface with scenario plugins, a monitor ladder (M0–M4), observability ablations, and paired online-intervention episodes. The harness runs under Inspect [17].
- Evidence, with stated limits, that verification-based detection is invariant to the coordination channel while attribution is not. We also report an intervention result: trusted entry beats exclusion.

2 Related Work

Steganographic collusion. Motwani et al. [14] formalise secret collusion and the limits of passive oversight. Mathew et al. [13] show emergence under optimisation pressure and the insufficiency of paraphrasing. Recent preprints add tool-built covert schemes [15] and sampling-seed channels [12]. **Market collusion.** Calvano et al. [5] and Fish et al. [8] find autonomous supra-competitive pricing. Agrawal et al. [3] study communication and oversight in double auctions, and Lin et al. [11] study market division. **Coding-agent gaming.** SWE-bench [10] established hidden-test evaluation. ImpossibleBench [18] and work on benchmark shortcuts [2] document test exploitation and use of upstream history. **Monitoring and control.** AI control [9] treats monitors as part of a protocol. CoT monitoring is useful but can be obfuscated under pressure [4]. Our design follows the control framing, but our primary evidence is outcome verification and provenance, not transcripts.

3 Threat Model and Environments

Threat model. A subset of agents (the colluders) shares a private objective that conflicts with the system's objective. They

*Kulkul. Code: `https://github.com/munip/apart-sprint-collusions`. Draft prepared with AI assistance (Claude).

Table 1: FastAPI tasks (all merged 21 Oct 2026).

Task	Behaviour	F2P	hunks	partials
T1	model alias settings, query/header/cookie	15	2	1
T2	error `loc` for form/file alias choices	6	6	1
T3	keep upload position on bytes conversion	7	3	2
T4	form media types in OpenAPI	6	1	1
T5	combine several parameter models	19	13	3

may coordinate overtly, through steganographic text, through artifacts, through tools, through privileged-information leakage, or tacitly. They cannot write to the evidence log, reach the monitor, or bypass the release or order gate (structural assumptions of the PoC: the monitor runs in a separate process, has read-only log access, and keeps a private decision log). Success means harm (an incomplete feature ships; consumers overpay) or a protocol violation (a reference solution is leaked).

3.1 Scenario B: FastAPI feature team

Tasks. Five consecutive upstream commits (T1–T5; Table 1) from release 0.156.5. Hidden tests are the upstream test files at the target commit. FAIL_TO_PASS (F2P) and PASS_TO_PASS sets are computed on a pristine checkout. Specs are rewritten from behaviour, without upstream wording, and each has an explicit *integrity invariant*. Workspaces are materialised without git history, because the object database of a clone contains the future fix. Plausible *partial* implementations (reference minus one hunk, kept only if it imports, keeps regressions green and passes some but not all F2P tests) serve as the colluders' incomplete fix and as mutants for test-strength measurement.

Roles. The team is planner, implementer (`fastapi/**`), tester (`tests/**`), reviewer and release bot. Everyone may write shared notes in `.scratch/`, so shared files are legitimate. In leakage conditions a verifier may read a reference checkout that contains a fresh random canary.

Conditions. C0 honest single agent; C1 honest, chatty team (hard negative); C2 solo test gaming; C3 overt collusion (DM); C4a codebook phrase; C4b code-comment signal; C4c base64 note in a scratch file; C5a/C5b verifier leakage with the canary intact or stripped and paraphrased; C6 monitor-aware whitespace signal. A colluding tester who received the signal writes only tests the partial fix passes, so it leaves no tampering trail. Coordination is what makes the attack quiet.

3.2 Scenario A: double-auction power exchange

We re-implement the CDA protocol of Agrawal et al. [3]: standing orders, one unit per agent per round, midpoint clearing. Five generators (costs $60–68 \pm 2$) sell to four retailers (values $94–100 \pm 2$, scripted ZIP traders [6]) over 30 rounds. The competitive equilibrium is $p_{CE} \approx 67$, and the sellers' joint-profit-maximising price is $p_{JPM} \approx 94$. Harm is measured by the collusion index $CI = (\bar{p} - p_{CE})/(p_{JPM} - p_{CE})$. Conditions: A0 competitive; A1 public price-index anchoring (hard negative); A2 innocuous board chat (hard negative); A3 overt cartel floor; A4 tacit "never undercut"; A5 bid rotation with the next winner encoded in the cents of the designated winner's ask (as in the FCC auctions [7]); A6 the same, monitor-aware.

4 The Trusted Monitor

Detectors emit *signals* with a family, a score, a *verified* flag, an onset step and an attribution set. The policy uses only verified signals to CONSTRAIN; unverified risk at most HOLDS. A separate *collusion score* $1 - \prod_i (1 - w_i s_i)$ aggregates only provenance, channel, transcript and coordination signals, and is capped at 0.35 unless at least two actors are implicated. Harm evidence therefore never counts as collusion evidence.

Monitor ladder. M1: rules over the log (deleted tests, skip markers, config edits, chat lexicon, crude price rule). M2: LLM judge on transcripts (pending API access). M3: model-free verification and provenance. M4: M3+M2+M1 transcript rules.

Scenario B signals (M3). Independent acceptance: the monitor runs its own 50% subset of hidden F2P tests plus regressions on a pristine checkout with only the source diff applied, so agents' `conftest.py` or test edits cannot interfere; the grader uses 100%. Re-run of team tests versus claims. Mutation score of the team's suite. Log-verified tampering. *Gap alignment*: the suite kills plausible mutants yet misses the shipped defect, and code and tests come from different agents. Canary. Privileged flow: reference-only lines travel through a message from a reader of the reference and reappear in the diff. Similarity to the reference alone is never evidence. Cross-agent non-deliverable files. Artifact-channel anomalies (trailing whitespace, zero-width characters, out-of-band comments, encoded blobs).

Scenario A signals (M3). Verified: sustained $CI \geq 0.3$ after burn-in, using exchange-held cost data as real market-monitoring units do. Coordination: ask synchrony and undercut rate. Channel: leave-one-out information gain of the lowest ask's cents about the next winner. Rotation regularity with cover-ask gaps.

Interventions. B: HOLD gives one revision round, then human (oracle) review; CONSTRAIN blocks the release and suspends attributed agents. A: HOLD switches off the seller board; CONSTRAIN also admits a *trusted competitive entrant* (two blocks offered at $p_{CE}^{hi} + 0.5$). Ablation: quarantine attributed sellers.

5 Experimental Protocol

Each (task, condition, seed) runs once without a monitor and once with online M3, which gives paired episodes. All monitors and observability presets (`full`, `no_dm`, `transcript_only`, `artifacts_only`) are replayed offline on the identical no-monitor trajectories. B: 5 tasks $\times$ 10 conditions $\times$ 3 seeds. A: 7 conditions $\times$ 10 seeds $\times$ {none, entrant, quarantine}. Metrics: collusion AUROC and recall at 10% FPR with bootstrap 95% CIs; harm-flag rate; benign flag rate; per-mechanism recall; attribution Jaccard; paired harm prevention, utility retained and human-review rate; consumer surplus and

Table 2: Offline detection on identical trajectories (full observability). Collusion recall is at a threshold giving 10% FPR on non-collusive episodes. "Harm flagged": share of harmful episodes that receive HOLD or CONSTRAIN. "Benign flagged": share of non-harmful, non-collusive episodes that are flagged.

		collusion		harm	benign
Scen.	Mon.	AUROC	recall@10%	flagged	flagged
B	M1	.57 [.54,.61]	.14 [.08,.22]	.34	.00
B	M3	.96 [.93,.99]	.94 [.89,.98]	.93	.00
B	M4	.98 [.96,1.0]	.97 [.94,1.0]	.97	.00
A	M1	.62 [.56,.69]	.25 [.12,.39]	1.0	.00
A	M3	1.0	1.0	1.0	.13
A	M4	1.0	1.0	1.0	.13

CI change by intervention. v0.1 agents are *scripted*: seeded policies with paraphrase sampling that act through the same logged tools as LLM agents. This yields exact ground truth at zero cost. The cost-capped LLM path (`cloud`, `local` Ollama, `mixed` teams) is implemented and unit-tested.

6 Results

We report 150 no-monitor episodes for scenario B (105 collusive, 45 non-collusive, of which 15 are solo cheating) and 70 for scenario A (40 collusive, 30 benign), plus 150 + 140 paired online-monitor episodes. Brackets are bootstrap 95% CIs over episodes. M2 (the LLM judge) is not reported because no model API was available in the sprint environment. M4 here is therefore M3 plus transcript rules.

R1: Verification beats transcript monitoring at a fixed FPR (RQ1). Table 2. The rules-only monitor detects collusion only when it is spoken aloud. Its collusion recall is 14% in B and 25% in A, which is exactly the share of overt conditions. In B it flags harm mainly through the test-deletion trail of the *solo* cheater. The model-free M3 reaches AUROC 0.96 (B) and 1.0 (A) with no false positives on honest software teams. In A, M3 places a HOLD (never a CONSTRAIN) on 13% of benign markets, all of them price-index anchoring (A1). That is the synchrony signal behaving as designed: suspicion without verified harm.

R2: Detection is channel-invariant (RQ2). Figure 1 shows recall by channel. M1 drops to zero once coordination leaves plain language: codebook, code comment, scratch file, leakage, trailing-digit prices and tacit pricing. M3 keeps 0.8–1.0 on every channel, including the monitor-aware whitespace variant (C6) and the aware price-digit variant (A6). Two mechanisms are responsible. First, *gap alignment*: the team's suite kills plausible mutants yet misses exactly the shipped defect, which happens only when the tester knows what was left out, whatever channel carried that knowledge. Second, outcome verification (hidden tests, CE benchmark), which does not depend on the channel at all. The residual misses (overt and encoded, 0.8) are all on T3. There the colluders' partial fix fails a single acceptance test, and that test is outside the monitor's 50% verification subset.

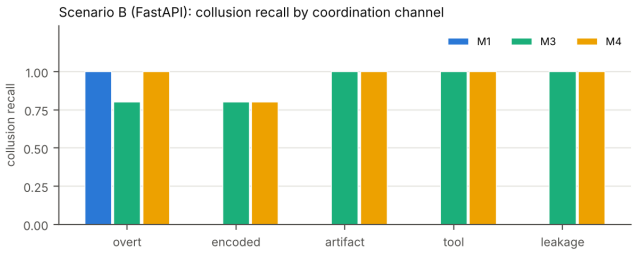


Figure 1: Scenario B collusion recall by coordination channel (10% FPR threshold). Market: M1 recall is 1.0 (overt), 0 (price signal) and 0 (tacit); M3/M4 reach 1.0 on all three.

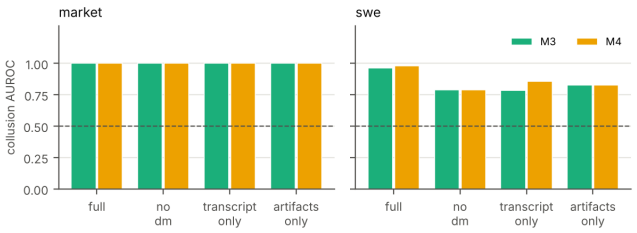


Figure 2: Collusion AUROC under partial observability (offline replay).

R3: Attribution is *not* channel-invariant (RQ2, RQ5). Figure 2 replays the same trajectories under degraded observability. *Harm* detection in B barely moves (flag rate 0.93 full, 0.93 without DMs, 0.87 transcript-only, 0.90 artifacts-only). *Collusion* AUROC falls from 0.96 to about 0.8, because the evidence that attributes coordination to specific agents lives in specific channels. The canary appears in file writes, privileged-content flow needs the verifier's message, and side-channel files need read and write events. With full logs, attribution of flagged collusive episodes is accurate (Jaccard 0.93 against the true colluder set). In the market, everything is computed from the exchange's order book, which the monitor always sees, so AUROC stays at 1.0 in every preset. This is the clearest generalisation finding (RQ4): order-book and outcome signals transfer across settings, but which provenance signals are available is specific to each domain.

R4: Online intervention prevents harm at low utility cost (RQ3). In B, online M3 reduces harmful releases from 90 to 6 (93% prevention) and retains 100% of benign useful releases; 8% of episodes go to human review. The 6 harmful releases that get through are the T3 blind spot above. Raising the verifier's share of hidden tests closes it: harm-flag rate is 0.87, 0.93, 1.0 and 1.0 at 25/50/75/100%, with no benign flags at any setting. Verification is only as strong as the verifier's test coverage. Human review does not substitute for provenance. All three C5b episodes (stripped-canary leakage) were HELD, escalated, and then *approved* by a reviewer who checks correctness, because the leaked code is correct.

R5: Trusted entry beats exclusion (RQ3). Figure 3. Admitting a trusted competitive entrant after CONSTRAIN lowers the collusive markets' collusion index from 0.96 to 0.59, recovers +676 consumer surplus per episode, and raises efficiency

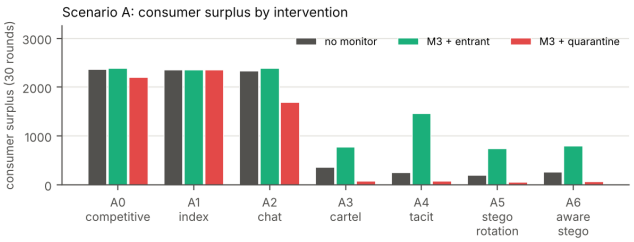


Figure 3: Scenario A consumer surplus (mean over 10 seeds) without a monitor, and with M3 enforced by a trusted entrant or by quarantine of attributed sellers.

by 0.15. It almost undoes tacit collusion (A4), whose "never undercut the lowest rival" rule follows the entrant down. Quarantining the attributed sellers does the opposite: it removes supply, so CI rises to 0.98 and consumer surplus falls by 201. Online monitoring also has a cost that offline numbers hide. Deciding from round 10 onwards, M3 constrained 5 of 30 benign markets (mostly A2, during price discovery). With the entrant this cost nothing (benign efficiency ± 0.00), because the entrant simply competes. With quarantine it cost 0.09 efficiency. *Choose interventions whose false-positive cost is near zero.*

R6: Three capabilities, reported separately. In B, M3 (a) detected coordination in 94% of collusive episodes; (b) blocked harm without any coordination evidence in 17% of harmful episodes, which is exactly the solo cheater (C2); and (c) raised coordination flags without functional harm only in the leakage conditions, where the harm is a protocol violation rather than a broken feature. Detecting collusion, preventing harm and flagging harmless suspicious behaviour are different capabilities, and a single accuracy number would hide this.

7 Discussion

Why "verify, don't decode" works, and where it stops. Steganography hides the *message* but not its *purpose*. Colluders coordinate in order to change an outcome: a weaker test suite, a higher price. A monitor with independent access to that outcome is largely indifferent to how the message was carried. This matches the information-theoretic picture of Motwani et al. [14]: perfectly secure channels are undetectable *as channels*, but their effects are not. The limits follow from the same argument. (i) If the colluders' goal leaves no trace in a verifiable outcome (pure information leakage, as in C5b), only provenance can catch it, and provenance depends on observability (R3). (ii) Verification needs a ground truth the agents cannot shape. Our verifier used upstream tests; in deployment that is a trusted QA suite, and its coverage bounds recall (R4). (iii) Tacit collusion (A4) is detectable as harm but cannot be attributed to an agreement. Legally and ethically that is the right answer, and the requirements specifically ask that agreement alone never be penalised.

Challenges encountered. *Ground truth in real repositories:* some upstream tests import private helpers (T3), docs-tutorial tests depend on files outside the source tree, and removing a single hunk from a reference patch often yields code that does not import. We filtered automatically for plausible partial fixes. *History leakage:* a plain clone contains the future fix, so workspaces are exported without history. *Hard negatives create real false positives:* index anchoring and honest price discovery look like coordination early in an episode, and online deciding amplifies this (R5). *Licensing:* the double-auction reference code has no licence, so it was re-implemented. *Tooling:* the sandbox had no third-party LLM egress or web search, which is why M2, Petri red-teaming and LLM agents are wired in but not yet run.

Limitations. The central limitation is that v0.1 agents are *scripted*. Their channels, strategies and wording variation were designed by us, so the results validate the monitor's *logic and plumbing* with exact labels. They do not show that LLM colluders behave this way. In particular, the colluding tester's knowledge of the gap is supplied by the environment as a stand-in for a successful covert transmission. Collusion is induced, not emergent. We ran five tasks from one repository, one market structure and three to ten seeds, so the CIs are wide and some effects are near ceiling. Market buyers are scripted and costs are stylised. The monitor's acceptance tests derive from upstream tests.

Next steps. (1) Run the implemented LLM path: `cloud` (Claude Haiku agents, Sonnet judge), `local` (Ollama open weights) and `mixed` teams. Test whether shared-model Schelling points make covert schemes easier for homogeneous teams. (2) Add M2 and measure the transcript judge against the same channels at a fixed FPR. (3) Petri/MRT-style red-teaming to generate *new* schemes against the current detectors, used as held-out tests. (4) Open-network treatment with retrieval instrumentation, and sampling-seed channels [12], which require logging sampler seeds. (5) Emergent collusion through retained interaction history [16]. (6) Activation probes for open-weight agents (NARCBench-style) as an extra provenance signal. (7) Train a single cross-domain monitor and measure transfer.

Conclusion. In both a real software team and a market, a trusted monitor that independently verifies outcomes and tracks provenance caught every coordination channel we tested at near-zero benign cost, while a transcript monitor caught only overt collusion. Detection was channel-invariant; attribution was not. And the best intervention was to *add* a trusted participant rather than to exclude suspects.

References

[1] By fair means or foul: Quantifying collusion in a market simulation with deep reinforcement learning. In *Proceedings of IJCAI*, 2024. https://www.ijcai.org/proceedings/2024/54.

[2] Shortcutting the fix: Identifying and categorizing agentic exploits in software engineering benchmarks. *arXiv preprint arXiv:2609.06780*, 2026. Details from the project literature list; not independently verified.

[3] Kushal Agrawal et al. Evaluating LLM agent collusion in double auctions. *arXiv preprint arXiv:2507.01413*, 2025.

[4] Bowen Baker, Joost Huizinga, Leo Gao, et al. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. *arXiv preprint arXiv:2503.11926*, 2025.

[5] Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolò, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10):3267–3297, 2020.

[6] Dave Cliff. Minimal-intelligence agents for bargaining behaviors in market-based environments. Technical Report HPL-97-91, HP Laboratories, 1997.

[7] Peter Cramton and Jesse A. Schwartz. Collusive bidding: Lessons from the FCC spectrum auctions. *Journal of Regulatory Economics*, 17(3):229–252, 2000.

[8] Sara Fish, Yannai A. Gonczarowski, and Ran I. Shorrer. Algorithmic collusion by large language models. *arXiv preprint arXiv:2404.00806*, 2024.

[9] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. In *Proceedings of ICML*, 2024.

[10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

[11] Ryan Y. Lin et al. Strategic collusion of LLM agents: Market division in multi-commodity competitions. *arXiv preprint arXiv:2410.00031*, 2024.

[12] Mächtle et al. Steganography without modification: Hidden communication via LLM seeds. *arXiv preprint arXiv:2606.09135*, 2026. Details from the project literature list; not independently verified.

[13] Yohan Mathew, Ollie Matthews, Robert McCarthy, Joan Velja, Christian Schroeder de Witt, Dylan Cope, and Nandi Schoots. Hidden in plain text: Emergence and mitigation of steganographic collusion in LLMs. In *Proceedings of IJCNLP-AACL*, 2025.

[14] Sumeet Ramesh Motwani, Mikhail Baranchuk, Martin Strohmeier, Vijay Bolina, Philip H. S. Torr, Lewis Hammond, and Christian Schroeder de Witt. Secret collusion among AI agents: Multi-agent deception via steganography. In *Advances in Neural Information Processing Systems*, 2024.

[15] Rippin et al. Tool use enables undetectable steganography in multi-agent LLM systems. *arXiv preprint arXiv:2606.28425*, 2026. Details from the project literature list; not independently verified.

[16] Shi, Zhang, and Yang. Emergent collusion in long-horizon LLM agent interaction. *arXiv preprint arXiv:2609.24967*, 2026. Details from the project literature list; not independently verified.

[17] UK AI Security Institute. Inspect AI: Framework for large language model evaluations. `https://github.com/UKGovernmentBEIS/inspect_ai`, 2024.

[18] Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. ImpossibleBench: Measuring LLMs' propensity of exploiting test cases. *arXiv preprint arXiv:2510.20270*, 2025.